

Rumi

Pre-development spec

FOUNDATIONS

Overview & scope

Key decisions

Architecture

Repo & folder structure

DATA

Database schema

Security & RLS

PRODUCT

Sitemap

Page-by-page spec

Shared components

Admin panel

INTEGRATIONS

Payments (Razorpay)

WhatsApp button

Images & storage

SHIP IT

Free-tier stack

Environment variables

Deployment steps

Real-world costs

Open questions

# Rumi — Build Spec

Everything a developer needs to build the Rumi candle store: a full frontend storefront, a Supabase-backed data layer, a private admin panel, on-site payments, and a WhatsApp channel — using only free-tier tools. Pair this with the **Rumi Brand Bible** for every color, font, and tone-of-voice decision; this document does not repeat those values, it applies them.

### What this covers

A candle e-commerce site with three parts, built as three folders in one repository: a public **storefront** (browse, cart, checkout, pay), a private **admin panel** (product upload, order management — access restricted to the owner only), and a **backend** that is entirely managed (Supabase), so there is no server to patch or pay for. A floating WhatsApp button sits on every storefront page for support and pre-sale questions, alongside full on-site checkout.

### Assumption stated up front

This spec assumes India as the primary market (INR pricing, Razorpay as the payment gateway, IST timing) based on context available about the business. If Rumi will sell outside India, flag it before development starts — it changes the payment gateway, currency handling, and shipping-zone logic.

### Key decisions this spec makes

These were the open forks in scope. Each is decided below so development can start without re-litigating them; each is also cheap to revisit later if it turns out wrong.

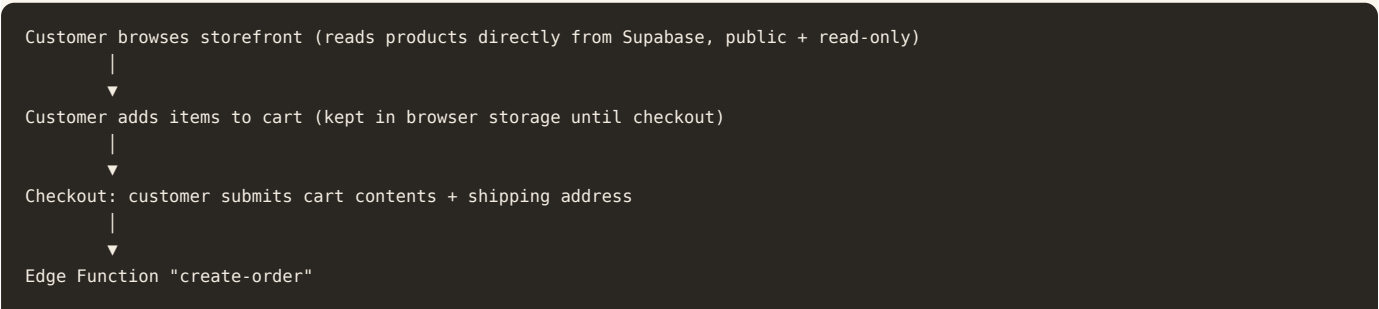
| DECISION         | CHOSEN                                                                           | WHY                                                                                                                                                                                                                                                             |
|------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Site model       | Full e-commerce (on-site cart, checkout, and payment) with a private admin panel | Confirmed as the desired scope. The WhatsApp button stays, but as a support/enquiry channel, not the only path to purchase.                                                                                                                                     |
| Hosting cost     | Zero recurring cost                                                              | Every tool below is on a free tier suitable for a small commercial site (see the caveat under <a href="#">Free-tier stack</a> on Cloudflare specifically). The one unavoidable real-world cost is a custom domain name (see <a href="#">Real-world costs</a> ). |
| Visual direction | Luxury & elegant, premium/gifting positioning                                    | Set in the Brand Bible; this document's components and admin UI follow the same tokens so the admin panel doesn't feel like a bolted-on generic dashboard.                                                                                                      |
| Backend model    | Backend-as-a-service (Supabase), not a hand-rolled Node/Express server           | Removes an entire category of hosting cost and maintenance (no server to keep patched or paying for) while still giving a real Postgres database, authentication, file storage, and serverless functions.                                                       |

### Architecture

Three moving pieces, no traditional backend server:

|                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Storefront</p> <p>Next.js (React) site. Reads products directly from Supabase (public, read-only). Writes — placing an order — go through one serverless Edge Function, never straight to the database, so prices can't be tampered with client-side.</p> | <p>Admin panel</p> <p>A second, separate Next.js app, deployed on its own subdomain (e.g. <code>admin.rumicandles.com</code>). Locked behind Supabase Auth, allow-listed to your email only. Full read/write to products, orders, and settings.</p> | <p>Backend</p> <p>Supabase: Postgres database, authentication, file storage for product photos, and Edge Functions for the two operations that must never trust the browser — creating an order total and verifying a payment.</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Request flow — placing an order



```

→ re-fetches each product's REAL price from the database (ignores any price the browser sent)
→ computes subtotal + shipping fee = total
→ creates a Razorpay order for that total
→ inserts an `orders` row with payment_status = 'pending'
    |
    ▼
Razorpay Checkout opens in the browser, customer pays
    |
    ▼
Edge Function "verify-payment" (called by the browser AND by Razorpay's webhook, belt-and-braces)
→ verifies Razorpay's signature server-side
→ on success: payment_status = 'paid', order_status = 'confirmed'
    |
    ▼
Order confirmation page + (optional) WhatsApp / email notification

```

### The one rule that matters most in this whole document

Never let the browser tell the backend what an order costs. Every price used to charge a customer must be re-read from the `products` table inside the Edge Function, not trusted from the cart the browser posted. This is the single most common security mistake in DIY e-commerce builds.

## Repository & folder structure

One Git repository, three top-level folders, matching how the pieces are described above and deployed independently:

```

rumi/
├── frontend/                # Public storefront – Next.js (App Router)
│   ├── app/
│   │   ├── page.tsx        # Home
│   │   ├── shop/
│   │   ├── product/[slug]/
│   │   ├── cart/
│   │   ├── checkout/
│   │   ├── order-confirmation/[orderNumber]/
│   │   ├── track-order/
│   │   ├── about/
│   │   ├── contact/
│   │   └── policies/
│   ├── components/        # Button, ProductCard, WhatsAppButton, Nav, Footer, CartDrawer...
│   ├── lib/
│   │   ├── supabaseClient.ts # anon key only – public, read-only calls
│   │   └── formatCurrency.ts
│   ├── styles/tokens.css   # generated from the Brand Bible's tokens.json – do not hand-edit values
│   └── public/
├── admin/                  # Private admin panel – separate Next.js app
│   ├── app/
│   │   ├── login/
│   │   ├── dashboard/
│   │   ├── products/      (list, /new, /[id]/edit)
│   │   ├── orders/        (list, /[id])
│   │   ├── categories/
│   │   └── settings/
│   ├── components/
│   └── lib/supabaseAdminClient.ts # still anon key + logged-in session; NEVER ships the service role key to any browser bundle
├── backend/                # Supabase project – the "backend", entirely config + SQL, no server to run
│   ├── migrations/
│   │   ├── 0001_init_schema.sql
│   │   ├── 0002_rls_policies.sql
│   │   └── 0003_seed_categories.sql
│   ├── functions/
│   │   ├── create-order/index.ts # Edge Function – recalculates totals, opens Razorpay order
│   │   ├── verify-payment/index.ts # Edge Function – verifies Razorpay signature, marks order paid
│   │   └── razorpay-webhook/index.ts # Edge Function – Razorpay's server-to-server callback
│   └── config.toml
└── docs/                  # This document, the Brand Bible export, and any decision notes

```

Deployment matches the folders: `frontend/` and `admin/` are each their own Cloudflare Pages project (two subdomains, two deploys), and `backend/`'s migrations and functions are pushed to Supabase with its CLI. Nothing about this requires a monorepo tool — plain folders and two build configs are enough at this scale.

## Database schema

Postgres, hosted by Supabase. Written as the actual SQL a developer runs as the first migration — copy, don't just paraphrase.

```
-- categories
create table categories (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text unique not null,
  description text,
  sort_order int default 0,
  created_at timestamptz default now()
);

-- products
create table products (
  id uuid primary key default gen_random_uuid(),
  category_id uuid references categories(id),
  name text not null,
  slug text unique not null,
  description text,
  scent_notes text[],          -- e.g. {"Amber", "Sandalwood", "Smoke"}
  ingredients text,
  burn_time_hours numeric,
  weight_grams numeric,
  price_inr numeric not null,
  compare_at_price_inr numeric, -- optional strike-through "was" price
  stock_quantity int not null default 0,
  is_active boolean not null default false, -- the admin panel's "go live" toggle
  is_featured boolean not null default false,
  image_paths text[] not null default '{}', -- paths in Supabase Storage, ordered
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- product_variants (optional – sizes like "Small / 120g", "Large / 250g")
create table product_variants (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  name text not null,
  price_inr numeric not null,
  stock_quantity int not null default 0,
  sku text unique
);

-- orders
create table orders (
  id uuid primary key default gen_random_uuid(),
  order_number text unique not null,          -- e.g. "RUMI-000123", generated by the Edge Function
  customer_name text not null,
  customer_email text not null,
  customer_phone text not null,
  shipping_address jsonb not null,            -- {line1, line2, city, state, pincode}
  subtotal_inr numeric not null,
  shipping_fee_inr numeric not null default 0,
  total_inr numeric not null,
  payment_status text not null default 'pending'
  check (payment_status in ('pending', 'paid', 'failed', 'refunded')),
  payment_gateway_order_id text,              -- Razorpay order id
  payment_gateway_payment_id text,            -- Razorpay payment id, once paid
  order_status text not null default 'placed'
  check (order_status in ('placed', 'confirmed', 'packed', 'shipped', 'delivered', 'cancelled')),
  courier_name text,
  tracking_number text,
  notes text,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- order_items – a snapshot of what was bought, so later price/name edits never change past orders
create table order_items (
  id uuid primary key default gen_random_uuid(),
  order_id uuid references orders(id) on delete cascade,
  product_id uuid references products(id),
  product_name text not null,
```

```
variant_name text,
unit_price_inr numeric not null,
quantity int not null,
line_total_inr numeric not null
);

-- site_settings — admin-editable, single-row-per-key config (WhatsApp number, shipping fee, banner text)
create table site_settings (
  key text primary key,
  value jsonb not null
);

-- admins — allow-list of who may sign in to the admin panel (usually exactly one row)
create table admins (
  id uuid primary key references auth.users(id),
  email text unique not null
);
```

Optional, add later rather than at launch: a `coupons` table (code, discount\_type, discount\_value, active, expires\_at) and a `reviews` table once there's order history to attach reviews to.

### Security & Row Level Security (RLS)

Supabase's database is reachable directly from the browser, so every table needs an explicit rule for who can read or write it — this is what actually keeps the store safe, not the frontend code. Enable RLS on every table, then:

| TABLE                                                                           | PUBLIC (ANON) CAN...                                                                                                                          | ADMIN CAN...      |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <code>categories</code> , <code>products</code> , <code>product_variants</code> | Read rows where <code>is_active = true</code> only                                                                                            | Full read/write   |
| <code>orders</code> , <code>order_items</code>                                  | No direct access at all — every write happens inside the <code>create-order</code> Edge Function, which uses the service role key server-side | Full read/write   |
| <code>site_settings</code>                                                      | Read only the handful of keys marked public (WhatsApp number, shipping fee, banner text)                                                      | Full read/write   |
| <code>admins</code>                                                             | No access                                                                                                                                     | Read own row only |

"Admin can" above means: a Postgres policy that checks `exists (select 1 from admins where id = auth.uid())`. There is deliberately no separate "staff" role for launch — one owner account is the whole admin allow-list; add roles later only if a second person genuinely needs access.

**Never expose the service role key**

Supabase issues two keys: an *anon* key (safe to ship in the frontend and admin panel bundles — RLS still applies) and a *service role* key (bypasses RLS entirely). The service role key belongs only inside the three Edge Functions in `backend/functions/`, set as a Supabase secret — never in frontend code, never in a public repo, never in an environment variable prefixed `NEXT_PUBLIC_`.

Sitemap

| Storefront (public)                            |                                                                       |
|------------------------------------------------|-----------------------------------------------------------------------|
| <code>/</code>                                 | Home                                                                  |
| <code>/shop</code>                             | All products, filterable                                              |
| <code>/shop/[category]</code>                  | One category                                                          |
| <code>/product/[slug]</code>                   | Product detail                                                        |
| <code>/cart</code>                             | Cart drawer's full-page equivalent                                    |
| <code>/checkout</code>                         | Address + payment                                                     |
| <code>/order-confirmation/[orderNumber]</code> | Thank-you page                                                        |
| <code>/track-order</code>                      | Look up an order by number + phone                                    |
| <code>/about</code>                            | Brand story                                                           |
| <code>/contact</code>                          | WhatsApp, email, simple form                                          |
| <code>/policies</code>                         | Shipping, returns, privacy, terms — required by Razorpay's KYC review |

| Admin panel (private)            |                                                |
|----------------------------------|------------------------------------------------|
| <code>/login</code>              | Email + password (Supabase Auth)               |
| <code>/dashboard</code>          | Today's orders, revenue, low-stock alerts      |
| <code>/products</code>           | List + search                                  |
| <code>/products/new</code>       | Upload a new candle                            |
| <code>/products/[id]/edit</code> | Edit, toggle live, delete                      |
| <code>/categories</code>         | Manage categories                              |
| <code>/orders</code>             | List, filter by status                         |
| <code>/orders/[id]</code>        | Update status, add tracking number             |
| <code>/settings</code>           | WhatsApp number, shipping fee, banner, socials |

Page-by-page spec

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Home                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Hero (full-bleed lit-candle photograph, <code>charcoal</code> gradient overlay for text legibility, the brand's hero line — see Brand Bible §3 — and one primary button in <code>gold</code> ). Below it: a featured-products grid (pulls <code>is_featured = true</code> ), a short brand-story block linking to <code>/about</code> , a "how it's made" strip (three photos: pouring, wick-trimming, wrapping), and the footer. No pop-ups, no countdown timers — see Brand Bible §11. |
| Shop / category                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| A responsive grid of <code>ProductCard</code> components (see §Components). Filters: category (chips), scent family, price range — kept simple (no faceted search infrastructure needed at this catalog size). Sort: featured, price low–high, price high–low, newest.                                                                                                                                                                                                                   |
| Product detail                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Image gallery (swipeable on mobile) on one side, on the other: name (display font), price, scent notes as small tags, burn time and weight as a compact spec row, variant picker (if variants exist), quantity stepper, <b>Add to Cart</b> (primary, gold) and <b>Ask on WhatsApp</b> (secondary, outlined) side by side. Below the fold: full description, ingredients, care instructions, and a "you may also like" row of 4 related products from the same category.                  |
| Cart                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Line items with thumbnail, name, variant, quantity stepper, remove. Subtotal, an estimated shipping note ("calculated at checkout"), and a single <b>Checkout</b> button. Empty state: a friendly line in brand voice, not a bare "Your cart is empty."                                                                                                                                                                                                                                  |
| Checkout                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| One page, three sections top to bottom: shipping address (name, phone, email, address, city, state, pincode), order summary (collapsed on mobile, expandable), and the payment button. Submitting calls the <code>create-order</code> Edge Function first (see §Payments) — the Razorpay Checkout modal only opens once that function returns a valid order id.                                                                                                                          |
| Order confirmation                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Order number, a checkmark moment (small, not a confetti animation — see Brand Bible §10 on motion), item summary, delivery estimate, and both a "Continue shopping" link and a WhatsApp link pre-filled with the order number for post-purchase questions.                                                                                                                                                                                                                               |
| Track order                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| A small form (order number + phone number) that calls a read-only Edge Function or a narrowly-scoped RLS policy (never a public "list all orders" query) and shows status, courier name, and tracking number if set.                                                                                                                                                                                                                                                                     |

Shared components

Built once, reused everywhere, styled from the Brand Bible's design tokens (never a hand-picked hex value in a component file):

| COMPONENT      | SPEC                                                                                                                                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Button         | Variants: primary ( gold fill, ink text), secondary (outline, wine border and text), ghost (text only). radius-full . 250ms ease transition on hover, per Brand Bible §10. |
| ProductCard    | sand background, radius-md , shadow-sm resting / shadow-md hover. Image, name, price, a small "sold out" badge in error when stock_quantity = 0 .                          |
| Nav            | paper background, logo left, links center, cart icon with item-count badge right. Collapses to a hamburger + slide-out panel under 880px.                                  |
| Footer         | charcoal background, charcoal-ink text. Sitemap links, policies links, social icons, and a small "hand-poured in [city]" line.                                             |
| WhatsAppButton | See §WhatsApp button below — its own section, since it has exact placement and behavior rules.                                                                             |
| CartDrawer     | Slides in from the right, shadow-lg , same line-item layout as the full cart page.                                                                                         |

Admin panel

A separate app so the public storefront's bundle never ships a single line of admin code. Access: Supabase Auth email/password login, gated by the admins table (see §Security) — there is no public sign-up screen anywhere in this app.

Product upload flow (the core admin job)

- Admin fills in name, category, description, scent notes, ingredients, burn time, weight, price, stock quantity.
- Admin uploads 1–6 photos, which go straight to Supabase Storage's product-images bucket from the browser (using the logged-in admin session — RLS on the storage bucket restricts writes to admins).
- The product saves with is\_active = false by default — a deliberate staging step.
- Admin reviews it on a "preview" link (the real product page, unlisted), then flips Go live — setting is\_active = true — when ready. This is the one control that actually makes a product visible to customers.

Order management

List view with filter chips for each order\_status . Opening an order shows the full item list, customer and shipping details, and payment status (read-only — payment status is only ever set by the verify-payment function, never edited by hand, to avoid an admin accidentally marking an unpaid order as paid). The admin can advance order\_status (placed → confirmed → packed → shipped → delivered) and add a courier name + tracking number, or cancel an order.

Dashboard

Four numbers above the fold: today's orders, today's revenue, orders awaiting packing, and products below a low-stock threshold (configurable in Settings). Kept to numbers a small operation actually checks each morning — resist the urge to build a full analytics suite here.

Payments — Razorpay

**Why Razorpay:** among the mainstream Indian gateways, it charges no setup fee and no annual maintenance charge — only a per-transaction fee (about 2% on standard domestic UPI/card payments as of late 2026; confirm the current rate on Razorpay's own pricing page before launch, since gateway pricing changes). Cashfree's lower headline rate is offset by a real annual fee; Instamojo is a reasonable alternative with a similar zero-AMC structure. This transaction fee is unavoidable for any real payment gateway and is the one "cost" inside an otherwise free stack — it scales with sales, not a fixed bill.

Before writing any payment code

Razorpay requires business KYC before it will process live payments: PAN, bank account details, and basic business proof (even a sole-proprietorship declaration is usually enough to start). This is a paperwork step for the business owner, not a technical one — start it early, since verification can take a few days, and build against Razorpay's test mode (fully free, no KYC needed) in the meantime.

Integration shape

- create-order** Edge Function: takes the cart from the browser, re-prices it from the products table, creates a Razorpay order via their server-side API, inserts the local orders row as pending , and returns the Razorpay order id to the browser.
- Browser opens Razorpay's Checkout widget with that order id — this is the only part of the integration that runs client-side, and it never sees a raw price it could tamper with.
- verify-payment** Edge Function: receives Razorpay's payment id + signature after checkout, verifies the signature server-side using the Razorpay secret key, and only then marks the order paid .
- razorpay-webhook** Edge Function: Razorpay also calls this directly (configured in the Razorpay dashboard) as a second, more reliable confirmation — covers the case where a customer closes the browser tab right after paying, before step 3 can run client-side.

## WhatsApp floating button

Zero-cost, no API key, no approval process: a plain `wa.me` link. Present on every storefront page (not the admin panel), fixed to the bottom-right corner.

| PROPERTY           | SPEC                                                                                                                                                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Link format        | <code>https://wa.me/&lt;number-with-country-code, no plus or spaces&gt;?text=&lt;url-encoded pre-filled message&gt;</code>                                                                                                                |
| Pre-filled message | Contextual — e.g. on a product page: <i>"Hi Rumi, I'd like to ask about the Amber Dusk candle."</i> On the general contact page: a plain <i>"Hi Rumi, I have a question."</i>                                                             |
| Position           | Fixed, bottom: 24px, right: 24px (16px on mobile, respecting the safe-area inset)                                                                                                                                                         |
| Style              | <code>radius-full</code> , <code>shadow-lg</code> . Resting state uses the brand's <code>gold</code> ring per Brand Bible §10; the WhatsApp glyph itself keeps WhatsApp's own green, per their brand guidelines — don't recolor the logo. |
| Number source      | Read from <code>site_settings</code> (key <code>whatsapp_number</code> ) so the admin can update it from <code>/admin/settings</code> without a code deploy.                                                                              |

## Images & storage

Product photos live in a Supabase Storage bucket ( `product-images` , public-read / admin-write), which fits comfortably in the 1 GB free-tier storage allowance for a candle catalog's photography. Serve images through Next.js's built-in image component so they're resized and lazy-loaded automatically; this alone removes most of the need for a separate image CDN. If the catalog grows past a few hundred products with many photos each, add Cloudinary's free tier (25 monthly credits — roughly 1,000 transformations, 1 GB storage, or 1 GB bandwidth, in any mix) purely for on-the-fly transformations; it's an upgrade path, not a launch requirement.

## Free-tier stack

Every service below has a free tier that explicitly allows commercial/business use — this was checked deliberately, because some popular hosts (notably Vercel's Hobby plan) restrict their free tier to personal, non-commercial projects. Limits below are current as of research for this document (September 2026) and are worth re-checking before launch, since providers do revise free-tier terms.

| NEED                                       | TOOL                              | FREE TIER                                                                                                                                                                         | COMMERCIAL USE           |
|--------------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| Storefront hosting                         | Cloudflare Pages                  | Unlimited sites & bandwidth, 500 builds/month, 100,000 Workers requests/day                                                                                                       | NO NON-COMMERCIAL CLAUSE |
| Admin panel hosting                        | Cloudflare Pages (second project) | Same as above                                                                                                                                                                     | NO NON-COMMERCIAL CLAUSE |
| Database + Auth + Storage + Edge Functions | Supabase                          | 500 MB database, 1 GB file storage, 50,000 monthly active users, 500,000 Edge Function calls/month; free projects pause after 7 days of inactivity (a scheduled ping avoids this) | ALLOWED                  |
| Payments                                   | Razorpay                          | No setup fee, no annual fee — ~2% per transaction (unavoidable, scales with sales, see §Costs)                                                                                    | BUSINESS KYC REQUIRED    |
| Customer contact                           | WhatsApp click-to-chat            | Free, no account or API needed                                                                                                                                                    | ALLOWED                  |
| Image transforms (optional, later)         | Cloudinary                        | 25 credits/month (≈1,000 transforms, or 1 GB storage, or 1 GB bandwidth)                                                                                                          | ALLOWED                  |
| Order/confirmation email (optional)        | Resend                            | 3,000 emails/month free tier (verify current limit before relying on it)                                                                                                          | ALLOWED                  |

Hosting note: Vercel is excellent for Next.js, but its own documentation states the free Hobby plan's fair-use terms restrict it to "non-commercial, personal use only" — a real risk for a store that takes payments. Cloudflare's published terms carry no equivalent line, and running a business site on its free plan is common practice, but Cloudflare has not published an explicit "commercial use: yes" statement either — re-read Cloudflare's current Pages terms before launch as a final check, rather than treating this document's read as the last word.

Environment variables

| VARIABLE                      | WHERE                                               | NOTES                                                         |
|-------------------------------|-----------------------------------------------------|---------------------------------------------------------------|
| NEXT_PUBLIC_SUPABASE_URL      | frontend, admin                                     | Safe to expose                                                |
| NEXT_PUBLIC_SUPABASE_ANON_KEY | frontend, admin                                     | Safe to expose — RLS is what actually protects data           |
| SUPABASE_SERVICE_ROLE_KEY     | Supabase Edge Function secrets only                 | Never in frontend or admin code, never committed to git       |
| RAZORPAY_KEY_ID               | Edge Function secrets + frontend (publishable half) | The key id alone is safe client-side; the secret below is not |
| RAZORPAY_KEY_SECRET           | Edge Function secrets only                          | Server-side signature verification only                       |
| RAZORPAY_WEBHOOK_SECRET       | Edge Function secrets only                          | Verifies the webhook payload actually came from Razorpay      |

Deployment steps

1. Create a free Supabase project. Run the migrations in `backend/migrations/` via the Supabase CLI or SQL editor.
2. Deploy the three Edge Functions ( `create-order` , `verify-payment` , `razorpay-webhook` ) with the Supabase CLI; set their secrets (service role key, Razorpay keys) in the Supabase dashboard, not in code.
3. Create a free Razorpay account, start in test mode, and note the test key id/secret for local development.
4. Push `frontend/` to a Git repository; connect it to a new Cloudflare Pages project; set its environment variables (the `NEXT_PUBLIC_*` ones) in the Pages dashboard.
5. Repeat for `admin/` as a second Cloudflare Pages project, on a separate subdomain.
6. Point a custom domain's DNS at Cloudflare Pages (both the root domain to the storefront and an `admin.` subdomain to the admin panel) once a domain is purchased.
7. Complete Razorpay's business KYC, switch from test keys to live keys, and do one real end-to-end test order before announcing launch.
8. Set up a free scheduled ping (e.g. a free cron service, or a Supabase-recommended GitHub Action) once a week against the Supabase project so it never auto-pauses from inactivity.

Real-world costs — the honest version

Everything above is genuinely free to run. Two things are not, and no configuration avoids them:

**REAL COST** A domain name

A `.com` or `.in` domain typically runs somewhere around ₹500–1,200/year depending on registrar and extension. The zero-cost fallback is a free subdomain Cloudflare Pages issues automatically (like `rumi.pages.dev`) — usable for launch and testing, just less polished for a premium brand.

**REAL COST** Payment gateway transaction fee

Razorpay's ~2% per transaction is not a subscription — it's a cut of each sale, unavoidable for any gateway that actually moves real money. It is the closest thing to a "cost" in this stack, and it only exists when a sale happens.

Open questions for the business side

Not blockers for starting development, but needed before launch — flagged here rather than guessed at:

- Final product catalog: exact scent list, sizes/variants, and pricing (this spec's schema supports variants; the actual list is a business decision).
- Shipping partner and shipping fee logic (flat rate vs. by weight/pincode) — affects the checkout page's shipping calculation.
- Return/refund policy wording, required for the `/policies` page and for Razorpay's KYC review.
- GST registration status — affects whether tax needs to be shown/collected at checkout.
- Business entity type for the Razorpay KYC paperwork (individual/sole proprietor vs. registered company).

Pair with the **Rumi Brand Bible** for the full color palette, type scale, voice, and imagery direction referenced throughout this document. Free-tier limits were verified via provider documentation and third-party trackers in September 2026 — reconfirm on each provider's own pricing page before committing, since these terms change over time.